

## ✓ Evaluation

```
from google.colab import drive
drive.mount('/content/drive')

!git clone https://github.com/prajwl7676/mini-clip-DLAI.git

%cd /content/mini-clip-DLAI
!pip install -q -r requirements.txt

import sys, os
sys.path.insert(0, '/content/mini-clip-DLAI')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force\_remount=True)  
fatal: destination path 'mini-clip-DLAI' already exists and is not an empty directory.  
/content/mini-clip-DLAI

```
===== 44.4/44.4 kB 4.0 MB/s eta 0:00:00
===== 61.0/61.0 kB 3.5 MB/s eta 0:00:00
===== 57.6/57.6 kB 4.1 MB/s eta 0:00:00
===== 906.4/906.4 MB 1.2 MB/s eta 0:00:00
===== 7.2/7.2 MB 110.0 MB/s eta 0:00:00
===== 9.7/9.7 MB 121.6 MB/s eta 0:00:00
===== 480.6/480.6 kB 43.9 MB/s eta 0:00:00
===== 4.5/4.5 MB 139.3 MB/s eta 0:00:00
===== 18.0/18.0 MB 80.0 MB/s eta 0:00:00
===== 8.3/8.3 MB 126.4 MB/s eta 0:00:00
===== 78.3/78.3 kB 9.0 MB/s eta 0:00:00
===== 12.9/12.9 MB 117.0 MB/s eta 0:00:00
===== 363.4/363.4 MB 4.7 MB/s eta 0:00:00
===== 13.8/13.8 MB 71.5 MB/s eta 0:00:00
===== 24.6/24.6 MB 36.8 MB/s eta 0:00:00
===== 883.7/883.7 kB 46.8 MB/s eta 0:00:00
===== 664.8/664.8 MB 881.0 kB/s eta 0:00:00
===== 211.5/211.5 MB 1.6 MB/s eta 0:00:00
===== 56.3/56.3 MB 15.1 MB/s eta 0:00:00
===== 127.9/127.9 MB 9.0 MB/s eta 0:00:00
===== 207.5/207.5 MB 7.0 MB/s eta 0:00:00
===== 188.7/188.7 MB 1.8 MB/s eta 0:00:00
===== 21.1/21.1 MB 83.1 MB/s eta 0:00:00
===== 99.1/99.1 kB 9.6 MB/s eta 0:00:00
===== 6.2/6.2 MB 117.2 MB/s eta 0:00:00
===== 209.6/209.6 MB 1.6 MB/s eta 0:00:00
===== 179.3/179.3 kB 18.4 MB/s eta 0:00:00
===== 566.4/566.4 kB 46.2 MB/s eta 0:00:00
===== 3.1/3.1 MB 87.8 MB/s eta 0:00:00
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is t  
opencv-contrib-python 4.13.0.92 requires numpy>=2; python\_version >= "3.9", but you have numpy 1.26.4 which is incompatible.  
tobler 0.14.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.  
tobler 0.14.0 requires tqdm>=4.67, but you have tqdm 4.66.6 which is incompatible.  
jaxlib 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.  
gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2024.9.0 which is incompatible.  
dataproc-spark-connect 1.1.0 requires tqdm>=4.67, but you have tqdm 4.66.6 which is incompatible.  
cupy-cuda12x 14.0.1 requires numpy<2.6,>=2.0, but you have numpy 1.26.4 which is incompatible.  
opencv-python 4.13.0.92 requires numpy>=2; python\_version >= "3.9", but you have numpy 1.26.4 which is incompatible.  
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.  
tiffiffle 2026.4.11 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.  
xarray-einstats 0.10.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.  
rasterio 1.5.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.  
shap 0.52.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.  
pytensor 2.38.3 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.  
opencv-python-headless 4.13.0.92 requires numpy>=2; python\_version >= "3.9", but you have numpy 1.26.4 which is incompatible.  
hdbscan 0.8.44 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.  
jax 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.

```
%cd /content/mini-clip-DLAI
```

```
/content/mini-clip-DLAI
```

```
import torch
import matplotlib.pyplot as plt
from datasets import load_dataset
from transformers import DistilBertTokenizer

from src.model_architecture import CLIPModel
```

```

from src.dataset import FlickrDataset, get_transform, build_loaders
from src.train import load_checkpoint
from src.evaluate import encode_dataset, recall_at_k, print_results, retrieval_examples

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')
BEST_CKPT = '/content/drive/MyDrive/mini-clip-dlai/checkpoints/best.pt'
EMBEDDING_DIM = 256
BATCH_SIZE = 64

```

Device: cuda

## ✓ Load data (test split only)

```

full_ds = load_dataset('nlphuji/flickr30k', split='test')
tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')

test_hf = full_ds.select(range(9_000, 10_000)) # same 1000 as training split
test_ds = FlickrDataset(test_hf, tokenizer, get_transform(train=False))
test_loader = torch.utils.data.DataLoader(
    test_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True
)
print(f'Test set: {len(test_ds)} samples → {len(test_loader)} batches')

```

/usr/local/lib/python3.12/dist-packages/huggingface\_hub/utils/\_auth.py:94: UserWarning:  
The secret `HF\_TOKEN` does not exist in your Colab secrets.  
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it  
You will be able to reuse this secret in all of your notebooks.  
Please note that authentication is recommended but still optional to access public models or datasets.

```

warnings.warn(
README.md: 100%                               641/641 [00:00<00:00, 14.5kB/s]

flickr30k.py:      2.51k/? [00:00<00:00, 141kB/s]

TEST/test/0000.parquet: 100%                    506M/506M [00:06<00:00, 78.9MB/s]
TEST/test/0001.parquet: 100%                    502M/502M [00:03<00:00, 267MB/s]
TEST/test/0002.parquet: 100%                    506M/506M [00:04<00:00, 131MB/s]
TEST/test/0003.parquet: 100%                    512M/512M [00:09<00:00, 50.2MB/s]
TEST/test/0004.parquet: 100%                    504M/504M [00:03<00:00, 140MB/s]
TEST/test/0005.parquet: 100%                    495M/495M [00:03<00:00, 156MB/s]
TEST/test/0006.parquet: 100%                    495M/495M [00:03<00:00, 185MB/s]
TEST/test/0007.parquet: 100%                    497M/497M [00:11<00:00, 45.3MB/s]
TEST/test/0008.parquet: 100%                    289M/289M [00:05<00:00, 47.5MB/s]

Generating test split: 100%                    31014/31014 [00:36<00:00, 1271.35 examples/s]

tokenizer_config.json: 100%                    48.0/48.0 [00:00<00:00, 3.67kB/s]

vocab.txt:      232k/? [00:00<00:00, 2.63MB/s]

tokenizer.json:   466k/? [00:00<00:00, 6.45MB/s]

config.json: 100%                               483/483 [00:00<00:00, 27.5kB/s]

Test set: 1000 samples → 16 batches

```

## ✓ Load best checkpoint

```

model = CLIPModel(embedding_dim=EMBEDDING_DIM).to(device)

projection_params = (
    list(model.image_encoder.projection.parameters()) +
    list(model.text_encoder.projection.parameters()) +
    [model.log_temperature]
)
encoder_params = [

```

```

    p for p in model.parameters()
    if p.requires_grad and not any(p is pp for pp in projection_params)
]

optimizer = torch.optim.AdamW([
    {'params': encoder_params, 'lr': 1e-4},
    {'params': projection_params, 'lr': 1e-3},
])

ckpt = load_checkpoint(model, optimizer, BEST_CKPT, device)

print(f'Loaded checkpoint from epoch {ckpt["epoch"] + 1}')
print(f'Val loss at best epoch : {ckpt["val_loss"]:.4f}')
print(f'Temperature  $\tau$  : {model.temperature.item():.4f}')

```

```

Loaded checkpoint from epoch 3
Val loss at best epoch : 1.6155
Temperature  $\tau$  : 0.0599

```

## ✓ Encode the full test set

```

print('Encoding test set...')
embeddings = encode_dataset(model, test_loader, device)

img_embs = embeddings['image_embeddings'] # (1000, 256)
txt_embs = embeddings['text_embeddings'] # (1000, 256)

print(f'Image embeddings : {img_embs.shape}')
print(f'Text embeddings : {txt_embs.shape}')

# Verify L2-normalisation
print(f'Image norms (should be ~1.0): mean={img_embs.norm(dim=1).mean():.4f}')
print(f'Text norms (should be ~1.0): mean={txt_embs.norm(dim=1).mean():.4f}')

```

Encoding test set...

Image embeddings : torch.Size([1000, 256])

Text embeddings : torch.Size([1000, 256])  
 Image norms (should be ~1.0): mean=1.0000  
 Text norms (should be ~1.0): mean=1.0000

## ✓ Compute Recall@K (fine-tuned model)

```

results_finetuned = recall_at_k(img_embs, txt_embs, ks=[1, 5, 10])
print_results(results_finetuned, title='Fine-tuned mini-CLIP (best.pt)')

```

Fine-tuned mini-CLIP (best.pt)

| Metric       | Score  |  |
|--------------|--------|--|
| Text → Image |        |  |
| R@1          | 17.10% |  |
| R@5          | 40.30% |  |
| R@10         | 54.60% |  |
| Image → Text |        |  |
| R@1          | 14.80% |  |
| R@5          | 41.20% |  |
| R@10         | 54.80% |  |

## ✓ Baseline: frozen pretrained encoders (no fine-tuning)

```

baseline_model = CLIPModel(embedding_dim=EMBEDDING_DIM).to(device)
baseline_model.eval()

print('Encoding test set with baseline (no fine-tuning)...')
base_embs = encode_dataset(baseline_model, test_loader, device)

results_baseline = recall_at_k(

```

```

base_embs['image_embeddings'],
base_embs['text_embeddings'],
ks=[1, 5, 10]
)
print_results(results_baseline, title='Baseline (frozen pretrained, no fine-tuning)')

```

Encoding test set with baseline (no fine-tuning)...

Baseline (frozen pretrained, no fine-tuning)

| Metric       | Score |
|--------------|-------|
| Text → Image |       |
| R@1          | 0.10% |
| R@5          | 0.60% |
| R@10         | 1.10% |
| Image → Text |       |
| R@1          | 0.10% |
| R@5          | 0.40% |
| R@10         | 1.20% |

## Results comparison table

```

print('\n=== Results Summary ===')
print(f'{"Metric":<14} {"Baseline":>10} {"Fine-tuned":>12} {"Δ gain":>8}')
print('-' * 48)
for direction, label in [("t2i", "Text→Image"), ("i2t", "Image→Text")]:
    print(f' {label}')
    for k in [1, 5, 10]:
        key = f'{direction}_R@{k}'
        base = results_baseline[key]
        fine = results_finetuned[key]
        gain = fine - base
        sign = '+' if gain >= 0 else '-'
        print(f'    R@{k:<4} {base:>9.2f}% {fine:>11.2f}% {sign}{gain:>6.2f}%')
print('-' * 48)

```

| === Results Summary === |          |            |          |
|-------------------------|----------|------------|----------|
| Metric                  | Baseline | Fine-tuned | Δ gain   |
| Text→Image              |          |            |          |
| R@1                     | 0.10%    | 17.10%     | + 17.00% |
| R@5                     | 0.60%    | 40.30%     | + 39.70% |
| R@10                    | 1.10%    | 54.60%     | + 53.50% |
| Image→Text              |          |            |          |
| R@1                     | 0.10%    | 14.80%     | + 14.70% |
| R@5                     | 0.40%    | 41.20%     | + 40.80% |
| R@10                    | 1.20%    | 54.80%     | + 53.60% |

## Plot results bar chart

```

import numpy as np

metrics = ['R@1', 'R@5', 'R@10']
ks = [1, 5, 10]

fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=False)

for ax, (direction, title) in zip(axes, [("t2i", "Text → Image"), ("i2t", "Image → Text")]):
    base_scores = [results_baseline[f'{direction}_R@{k}'] for k in ks]
    fine_scores = [results_finetuned[f'{direction}_R@{k}'] for k in ks]

    x = np.arange(len(ks))
    width = 0.35

    bars1 = ax.bar(x - width/2, base_scores, width,
                   label='Baseline', color='#B0BEC5', edgecolor='white')
    bars2 = ax.bar(x + width/2, fine_scores, width,
                   label='Fine-tuned', color='#1D9E75', edgecolor='white')

```

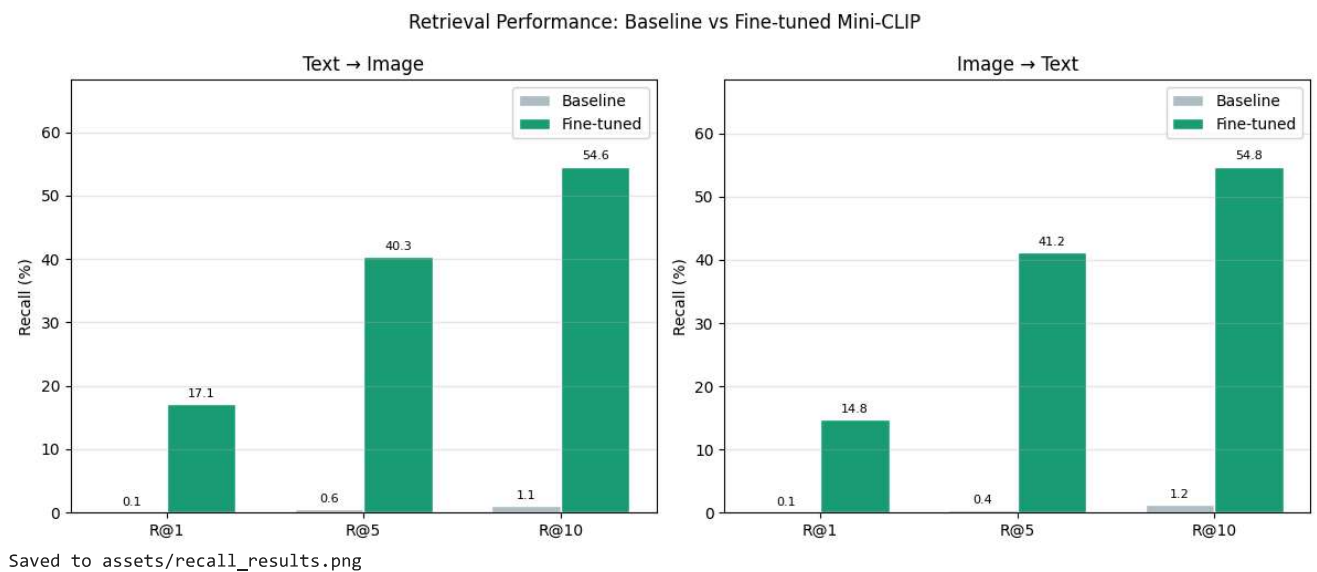
```

ax.set_xticks(x)
ax.set_xticklabels(metrics)
ax.set_ylabel('Recall (%)')
ax.set_title(title)
ax.legend()
ax.grid(True, axis='y', alpha=0.3)
ax.set_ylim(0, max(fine_scores) * 1.25)

for bar in bars1 + bars2:
    ax.annotate(
        f'{bar.get_height():.1f}',
        xy=(bar.get_x() + bar.get_width() / 2, bar.get_height()),
        xytext=(0, 3), textcoords='offset points',
        ha='center', va='bottom', fontsize=8
    )

plt.suptitle('Retrieval Performance: Baseline vs Fine-tuned Mini-CLIP', fontsize=12)
plt.tight_layout()
plt.savefig('assets/recall_results.png', dpi=120, bbox_inches='tight')
plt.show()
print('Saved to assets/recall_results.png')

```



## Qualitative retrieval examples

```

retrieval_examples(
    model = model,
    test_hf_dataset = test_hf,
    tokenizer = tokenizer,
    get_transform_fn = get_transform,
    device = device,
    query_indices = [0, 1, 2, 3],
    top_k = 5,
    save_path = 'assets/retrieval_examples.png',
)

```

Saved to assets/retrieval\_examples.png

Text → Image Retrieval — Top-5 results  
(green border = correct match)

Query 1:  
"A group of people watch a drag race at the beginning and see the"



Query 2:  
"A person has a mask, which has a red nose and a black bandan"



Query 3:  
"The picture takes place inside a large, commercial kitchen where people are"



Query 4:  
"An elderly Asian man is looking at a camera in a large gathe"

